Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Laboratoriya işi № 1

Tələbənin adı və soyadı: Fatimə Quliyeva

Qrup: KOM25

Müəllim: Şamil Hümbətov

Tarix: 21 oktyabr 2026

2026

1. Giriş

Bu laboratoriya işində üç əsas xətti verilənlər strukturu üzərində işlədim: Linked List, Stack və Queue. Hər struktur üçün real həyatdan bir ssenari götürüldü: musiqi playlisti (Linked List), brauzer tarixçəsi (Stack) və hava limanında qeydiyyat növbəsi (Queue). Strukturların hamısını Python dilində, hazır konteynerlərdən (collections.deque, queue.Queue və s.) istifadə etmədən, öz Node siniflərimlə və göstərici (reference) əlaqələri ilə sıfırdan yazdım.

İşin məqsədi yalnız kod yazmaq deyil, həm də əməliyyatların icra vaxtını ölçüb Big-O nəzəriyyəsi ilə müqayisə etməkdir. Hər struktur üçün 100, 1000 və 10000 element ilə test apardım, nəticələri cədvəl və qrafiklərlə təqdim etdim. Nəzəri əsas kimi Mark Allen Weiss-in *Data Structures and Algorithm Analysis in C++* kitabından istifadə etdim.

Sistem məlumatları

Testlər OneCompiler (onecompiler.com/python) online platformasında aparılıb. Bu səbəbdən aşağıda ölçmə mühiti, yəni online server göstərilir. Kodu yazdığım və nəticələri izlədiyim cihazın məlumatları isə ayrıca verilir.

Ölçmə mühiti (OneCompiler serveri)

Parametr	Dəyər
Platforma	OneCompiler, online Python mühiti
Dil / Interpreter	Python 3.12.14 (GCC 14.2.0 ilə qurulub)
Operating System	Linux 6.8.0-1067-azure (aarch64), glibc 2.41
CPU	ARM64 (aarch64) arxitekturalı server, 4 məntiqi nüvə
RAM / GPU	Saytda göstərilmir
Storage / Power Mode	Serverə aid deyil, saytda göstərilmir

Parametr	Dəyər
IDE	OneCompiler online editoru
Other Factors	Server paylaşılan mühitdir (başqa istifadəçilərlə), internet bağlantısı, hər işə salmada vaxtlar dəyişə bilər

İstifadə etdiyim cihaz

Parametr	Dəyər
Computer Model	HP Pavilion Laptop 15-eg3xxx
CPU	13th Gen Intel Core i5-1335U
RAM	8 GB
GPU	Intel Iris Xe Graphics
Operating System	Windows 11 Home 64-bit (Build 26200)
Screen Resolution	1920 × 1080 (scale 125%)
Storage	NVMe SSD
Power Mode	Balanced
Other Factors	Brauzer üzərindən işlədiyim üçün ölçmə nəticələrinə təsir etmir

Parametrlərin nəticələrə təsiri 8-ci bölmədə (Analiz) izah olunur.

---

2. Tapşırıq 1 – Smart Music Playlist (Linked List)

A. Araşdırma

1. Array və Linked List arasında əsas fərqlər hansılardır?
Array elementləri yaddaşda ardıcıl yerləşdirir, ona görə indeks ilə istənilən elementə birbaşa, O(1) vaxtda çatmaq olur. Linked List-də isə hər element ayrıca node-dur və node-lar yaddaşın müxtəlif yerlərində ola bilər. Onları bir-birinə göstəricilər bağlayır. Array-in ölçüsü adətən əvvəlcədən bəlli olur və

dolduqda genişləndirmə üçün bütün elementlər yeni yerə köçürülməlidir. Linked List isə dinamik böyüyür, hər yeni node üçün ayrıca yaddaş ayrılır. Array-in ortasına element əlavə etdikdə sonrakı elementləri sürüşdürmək lazım gəlir (O(n)), Linked List-də isə yeri tapdıqdan sonra yalnız iki göstəricini dəyişmək kifayətdir. Əvəzində Linked List-də hər node əlavə yaddaş (next göstəricisi) tələb edir.

2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?
Playlist tez-tez dəyişən strukturdur: istifadəçi mahnı əlavə edir, silir, ortadan bir mahnı çıxarır. Mahnıların sayı əvvəlcədən bəlli deyil, buna görə dinamik ölçü üstünlük verir. Mahnıların oxunması da adətən ardıcıl gedir (növbəti mahnı), bu isə Linked List-in next göstəricisinə təbii uyğun gəlir. Silmə və əlavə zamanı elementləri sürüşdürmək lazım olmur.

3. Singly və Doubly Linked List arasında üstünlüklər və çatışmazlıqlar
Singly Linked List-də hər node yalnız növbəti node-u göstərir. Yaddaş az sərf olunur və implementasiya sadədir, amma yalnız irəliyə hərəkət etmək olur. Bir node-u silmək üçün onun əvvəlkisini tapmaq lazım gəlir. Doubly Linked List-də həm next, həm də prev göstəricisi var. Hər iki istiqamətdə hərəkət etmək və silmək asanlaşır, lakin hər node-da əlavə göstərici üçün yaddaş tələb olunur və əlavə/silmə zamanı daha çox göstərici yenilənməlidir, bu da səhv ehtimalını artırır.

4. Linked List-də axtarış niyə Array-dən yavaş ola bilər?
Linked List-də indeks ilə birbaşa keçid yoxdur. Axtarılan elementə çatmaq üçün baş node-dan başlayıb hər node-a ardıcıl baxmaq lazımdır, bu da O(n) vaxt tələb edir. Əlavə olaraq, node-lar yaddaşda səpələnmiş olduğuna görə CPU keşindən səmərəli istifadə olunmur. Array isə ardıcıl yerləşdiyi üçün keşə daha uyğundur və praktikada daha sürətli işləyir.

5. Mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?
Linked List daha əlverişlidir. Sıranı dəyişmək üçün node-ları fiziki olaraq köçürmək lazım deyil, yalnız göstəriciləri yenidən bağlamaq kifayətdir. Array-

də isə sıra dəyişdikdə elementlər yerini dəyişməlidir. Doubly Linked List bu işi daha da asanlaşdırır.

B. Proqramlaşdırma

```
class SongNode:  # bir mahnı = zəncirin bir düyünü
    def __init__(self, song_id, name, artist):  # düyün yarananda 3 məlumat gəlir
        self.song_id = song_id  # mahnının unikal nömrəsi (1, 2, 3...)
        self.name = name       # mahnının adı
        self.artist = artist    # ifaçının adı
        self.next = None       # sağdakı (növbəti) düyün; hələ yoxdursa boş
        self.prev = None       # soldakı (əvvəlki) düyün; Doubly Linked List üçün



class Playlist:  # bütün mahnı zəncirini idarə edən sinif
    def __init__(self):  # boş playlist yaranır
        self.head = None  # birinci mahnı yoxdur
        self.tail = None  # sonuncu mahnı yoxdur
        self.size = 0     # mahnı sayı; Count üçün sayğac
        self.next_id = 1  # növbəti mahnıya veriləcək ID

    def is_empty(self):  # playlist boşdurmu?
        return self.head is None  # head yoxdursa içi boşdur

    def add_song(self, name, artist):  # Insert: mahnını sona əlavə et (O(1))
        song_id = self.next_id  # bu mahnıya növbəti boş ID-ni ver
        self.next_id += 1      # gələcək mahnı üçün ID-ni 1 artır
        new_node = SongNode(song_id, name, artist)  # təzə qutu; hələ zəncirdə deyil
        if self.is_empty():  # heç nə yoxdursa
            self.head = new_node  # bu düyün birinci olur
            self.tail = new_node  # eyni düyün sonuncu da olur
        else:  # artıq ən azı 1 mahnı varsa
            new_node.prev = self.tail  # yeninin solu = indiki sonuncu
            self.tail.next = new_node  # köhnə sonun sağı = yeni düyün
            self.tail = new_node      # "sonuncu" etiketi yeni düyünə keçir
```

```
    self.size += 1  # mahnı sayı 1 artır
    return song_id  # verilən ID-ni qaytar (axtarış/silmə üçün lazımdır)

def _find_by_id(self, song_id):  # daxili axtarış: başdan sona gəz (O(n))
    current = self.head  # birinci düyündən başla
    while current is not None:  # None = zəncir bitdi
        if current.song_id == song_id:  # ID uyğun gəldimi?
            return current  # həmin düyünü qaytar
        current = current.next  # yoxsa bir addım sağa keç
    return None  # axıra qədər tapılmadı

def search_song(self, song_id):  # Search: tapsa məlumat, tapmasa None
    node = self._find_by_id(song_id)  # əvvəl düyünü tap
    if node is None:  # belə ID yoxdursa
        return None  # nəticə yoxdur
    return (node.song_id, node.name, node.artist)  # tapıldısa qaytar

def delete_song(self, song_id):  # Delete: tapmaq O(n), qoparmaq O(1)
    node = self._find_by_id(song_id)  # silinəcək düyünü tap
    if node is None:  # belə ID yoxdursa
        return False  # heç nə silinmədi
    if node.prev is not None:  # solda kim varsa
        node.prev.next = node.next  # sol, silinəni ötüb birbaşa sağa baxsın
    else:  # solda heç kim yoxdursa bu head-dir
        self.head = node.next  # yeni birinci = sağdakı
    if node.next is not None:  # sağda kim varsa
        node.next.prev = node.prev  # sağ, birbaşa sola baxsın
    else:  # sağda heç kim yoxdursa bu tail-dir
        self.tail = node.prev  # yeni sonuncu = soldakı
    node.next = None  # silinən düyünü zəncirdən tam ayır
    node.prev = None  # geriyə də bağ qalmasın
    self.size -= 1    # say 1 azalır
    return True       # tapıldı və silindi
```

```
  def display_playlist(self):  # Display: bütün mahnıları ekrana yaz
    if self.is_empty():  # boşdursa
      print("Playlist boşdur.")  # mesaj ver
      return  # funksiyadan çıx
    print("PLAYLIST")  # başlıq
    index = 1  # ekranda 1, 2, 3... (massiv indeksi deyil, sadəcə nömrə)
    current = self.head  # başdan başla
    while current is not None:  # sona qədər
      print(f"{index}. [ID:{current.song_id}] {current.name} - {current.artist}")  # bir sətir çap et
      current = current.next  # növbəti düyünə keç
      index += 1  # ekran nömrəsini artır
    print("Cemi mahni:", self.count_songs())  # Count Songs


  def count_songs(self):  # Count Songs
    return self.size  # sayğacı qaytarırıq



p = Playlist()  # boş playlist yarat
p.add_song("Die for you", "The Weeknd")  # 1-ci mahnı, ID=1, sona
p.add_song("Slow Nights", "Tomoko Aran")       # 2-ci mahnı, ID=2, sona
p.add_song("Dracula", "Tame Impala")     # 3-cü mahnı, ID=3, sona
p.display_playlist()  # 3 mahnını göstər
print("Axtaris ID=3:", p.search_song(3))  # ID=3-ü tap və çap et
p.delete_song(3)  # ID=3-ü sil (ortadakı düyün qoparılır)
p.display_playlist()  # qalan 2 mahnını göstər
print("Say:", p.count_songs())  # neçə mahnı qaldığını yaz (2)
```

C. Təcrübə və Eksperiment

Element sayi	Insert (ms)	Search (ms)	Delete (ms)
100	0.073092	0.003640002	0.002935005
1000	0.748741004	0.024966997	0.013174998
10000	8.004153991	0.243122995	0.121497011

D. Analiz və Nəticələrin Şərhi

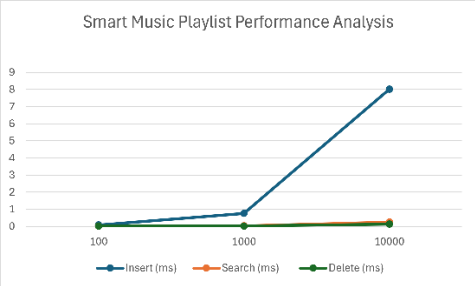
1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?
Mütləq vaxt baxımından ən çox Insert artdı: 0,073 ms-dən 8,004 ms-ə, yəni təxminən 110 dəfə. Bu, Insert-in n sayda əlavənin ümumi vaxtı olmasından qaynaqlanır. Bir əlavənin orta vaxtı isə 0,00073, 0,00075 və 0,00080 ms-dir, yəni demək olar ki, sabit qalıb, çünki tail göstəricisi saxlanılır və hər dəfə sona qədər gəzmək lazım olmur. Tək əməliyyat kimi ölçülən Search və Delete isə element sayı artdıqca açıq şəkildə təsirləndi, çünki hər ikisi axtarılan node-u tapmaq üçün siyahını gəzməlidir. Element sayı 100-dən 10000-ə qalxdıqda Search müddəti 0,0036 ms-dən 0,243 ms-ə (təxminən 67 dəfə), Delete müddəti isə 0,0029 ms-dən 0,121 ms-ə (təxminən 41 dəfə) artdı.

2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?
Böyük ölçüdə bəli. Nəzəriyyəyə görə Search və Delete O(n)-dir, yəni element sayı 10 dəfə artanda vaxt təxminən 10 dəfə artmalıdır. Mənim nəticələrimdə 1000-dən 10000-ə keçəndə Search vaxtı təxminən 9,7 dəfə, Delete vaxtı təxminən 9,2 dəfə artdı, bu da xətti artıma çox yaxındır. Insert-in ümumi vaxtı da təxminən 10 dəfə artdı (0,749 ms-dən 8,004 ms-ə), yəni bir əlavənin vaxtı sabit qaldı və bu, O(1) ilə üst-üstə düşür. Kiçik ölçülərdə (100 element) vaxt çox qısa olduğundan ölçmə xətası böyük rol oynayır və nəzəriyyədən kiçik sapmalar görünə bilər. Bu sapmalar keş effektindən, paylaşılan serverin yükündən və Python interpretatorunun əlavə xərclərindən qaynaqlana bilər.

3. Eyni problemi Array ilə həll etsəydim nəticələr necə olardı?
Array ilə ardıcıl axtarış yenə O(n) olardı, lakin ardıcıl yaddaş və keş uyğunluğuna görə praktikada Linked List-dən sürətli işləyərdi. İndeks ilə çıxarma O(1) olardı. Sonda əlavə orta hesabla O(1), ortadan əlavə və silmə isə elementlərin sürüşdürülməsinə görə O(n) olardı. Deməli, Array oxuma və axtarışda, Linked List isə tez-tez əlavə və silmədə üstündür.

4. Linked List-in əsas üstünlüyü nədir?
Əsas üstünlük dinamik ölçü və node-un yeri məlum olduqda O(1) əlavə və silmədir. Elementləri sürüşdürmək və ya bütün strukturu köçürmək lazım gəlmir.

5. Playlist tətbiqində Doubly Linked List daha məntiqli olardımı?
Mənim implementasiyam artıq Doubly Linked List-dir, çünki hər node-da həm next, həm də prev göstəricisi var. Bu, playlist üçün məntiqli seçimdir: “növbəti” və “əvvəlki” mahnı düymələri asanlıqla həyata keçirilir, silmə zamanı isə əvvəlki node-u axtarmaq lazım gəlmir (node tapıldıqdan sonra silmə O(1)-dir). Əvəzində hər node üçün əlavə prev göstəricisi yaddaş xərcləyir. Playlistlər adətən böyük olmadığı üçün bu əlavə xərc əhəmiyyətsizdir.**Qrafik**



3. Tapşırıq 2 – Browser History Manager (Stack)

A. Araşdırma

1. Stack strukturunda LIFO prinsipi nədir?
LIFO (Last In, First Out) “son daxil olan, ilk çıxır” deməkdir. Stack-də element yalnız bir uc tərəfdən, yəni “top”-dan əlavə edilir və çıxarılır. Ən son əlavə edilən element ilk çıxarılır. Bunu üst-üstə qoyulmuş boşqablar yığınına bənzətmək olar: boşqabı yalnız ən üstdən götürmək olar.

2. Browser History sistemi nə üçün Stack istifadə edir?
İstifadəçi səhifələrə ardıcıl keçir və “Back” basdıqda ən son baxdığı səhifədən əvvəlkinə qayıtmaq istəyir. Yəni ən son ziyarət edilən səhifə ilk geri qaytarılmalıdır, bu da LIFO-dur. Hər yeni səhifə stack-ə push olunur, Back basıldıqda isə pop olunur.

3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?
Hər edilən əməliyyat (mətn yazmaq, şəkil əlavə etmək və s.) stack-ə push olunur. Undo basıldıqda top-dakı əməliyyat pop edilir və geri qaytarılır. Beləliklə ən son edilən dəyişiklik ilk ləğv olunur, bu da istifadəçinin gözlədiyi davranışdır.

4. Queue istifadə olunsaydı hansı problemlər yaranardı?

Queue FIFO prinsipi ilə işləyir, yəni ilk daxil olan ilk çıxır. Bu halda "Back" basdıqda ən son səhifə yox, ən birinci ziyarət edilən səhifəyə qayıdılardı. Bu, istifadəçi gözləntisinə ziddir. Undo üçün də eyni problem olardı: ən son əməliyyat yox, ən köhnəsi ləğv edilərdi.

5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

Array əsaslı stack-də elementlər ardıcıl yaddaşda saxlanılır, push və pop sadədir və keşə uyğun olduğu üçün praktikada sürətlidir. Çatışmazlığı sabit ölçüdür: dolduqda overflow olur və ya array genişləndirilməlidir (bu zaman bir push O(n) ola bilər). Linked List əsaslı stack-də ölçü dinamikdir, hər push yeni node yaradır və overflow problemi yoxdur. Hər push və pop həmişə O(1)-dir (head-ə əlavə və head-dən çıxarma). Çatışmazlığı hər node üçün əlavə göstərici yaddaşı və node-ların yaddaşa səpələnməsidir.

B. Proqramlaşdırma

```
class Page:  # bir sehifenin melumatlarini saxlayan sinif
    def __init__(self, page_id, url, visit_time):  # sehife yaradilanda 3 deyer gelir
        self.page_id = page_id  # sehifenin nomresi
        self.url = url  # sehifenin unvani
        self.visit_time = visit_time  # ziyaret vaxti (istifadeci yazar)



class Node:  # stack-in bir qutusu
    def __init__(self, data):  # qutunun icine Page qoyulur
        self.data = data  # bu qutuda saxlanan sehife
        self.next = None  # altdaki qutuya ox; evvelce yoxdur



class Stack:  # oz stack-imiz
    def __init__(self):  # bos stack yaradilir
        self.top = None  # ustde hec ne yoxdur


    def is_empty(self):  # stack bosdurmu?
```

```
    return self.top is None  # top None-dursa bosdur


  def push(self, page):  # yeni sehifeni uste qoy (Visit Page), O(1)
    new_node = Node(page)  # sehife ucun yeni qutu
    new_node.next = self.top  # yeni qutunun alti = kohne ust
    self.top = new_node  # indi ust yeni qutudur


  def pop(self):  # ustdeki sehifeni cixar (Back), O(1)
    if self.is_empty():  # cixarilacaq sehife yoxdursa
      return None  # underflow: hec ne qaytarma
    page = self.top.data  # ustdeki sehifeni yadda saxla
    self.top = self.top.next  # ustu bir asagi endir
    return page  # cixan sehifeni qaytar


  def peek(self):  # uste bax, silme (Current Page), O(1)
    if self.is_empty():  # sehife yoxdursa
      return None  # hec ne yoxdur
    return self.top.data  # ustdeki sehifeni qaytar


  def display(self):  # tarixceni ustden asagi goster, pop etme
    if self.is_empty():  # bosdursa
      print("Tarixce bosdur.")  # mesaj yaz
      return  # funksiyani bitir
    current = self.top  # gezmeye ustden basla
    while current is not None:  # qutu bitene qeder
      p = current.data  # bu qutudaki sehife
      print("Page ID:", p.page_id)  # ID-ni cap et
      print("URL:", p.url)  # URL-i cap et
      print("Visit Time:", p.visit_time)  # vaxti cap et
      print("---")  # sehifeleri ayir
      current = current.next  # altdaki (daha kohne) qutuya kec



def print_page(page):  # bir sehifeni ekrana yaz
```

```
    print("Page ID:", page.page_id)  # ID
    print("URL:", page.url)  # URL
    print("Visit Time:", page.visit_time)  # Visit Time



def main():  # proqramin menyusu
    history = Stack()  # bos brauzer tarixcesi

    while True:  # Exit deyene qeder tekrar
        print("\n1. Visit Page")  # 1 = push
        print("2. Back")  # 2 = pop
        print("3. Current Page")  # 3 = peek
        print("4. Display History")  # 4 = hamisini goster
        print("5. Exit")  # 5 = cix
        secim = input("Secim: ")  # istifadeci nomre yazar

        if secim == "1":  # Visit Page
            page_id = input("Page ID: ")  # muellimin isteyen 1-ci sahe
            url = input("URL: ")  # 2-ci sahe
            visit_time = input("Visit Time: ")  # 3-cu sahe (saat avtomatik deyil)
            page = Page(page_id, url, visit_time)  # 3 saheli sehife yarat
            history.push(page)  # stack-in ustune qoy
            print("Sehife elave olundu.")  # tesdiq

        elif secim == "2":  # Back
            page = history.pop()  # ustdeki sehifeni cixar
            if page is None:  # stack bos idise
                print("Geri gedilecek sehife yoxdur.")  # underflow
            else:  # bir sehife cixdisa
                print("Back edilen sehife:")  # hansi baglandi
                print_page(page)  # o sehifenin 3 sahesini yaz

        elif secim == "3":  # Current Page
            page = history.peek()  # uste bax, silme
```

```
        if page is None:  # bosdursa
            print("Cari sehife yoxdur.")
        else:  # sehife varsa
            print("Cari sehife:")
            print_page(page)  # ID, URL, Visit Time

    elif secim == "4":  # Display History
        history.display()  # stack-i pozmadan goster

    elif secim == "5":  # Exit
        print("Cixilir.")  # mesaj
        break  # while-dan cix, proqram bitsin

    else:  # 1-5 deyil
        print("Yanlis secim.")  # xeta


if __name__ == "__main__":  # bu fayl birbasa ise dusende
    main()  # menyunu baslat
```

C. Təcrübə və Eksperiment

Emeliyyat sayi	Push (ms)	Pop (ms)
100	0.0339	0.0136
1000	0.2776	0.1299
10000	3.3025	1.3483

D. Analiz və Nəticələrin Şərhi

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?

Bəli, fərq müşahidə olundu. Push hər ölçüdə Pop-dan təxminən 2-2,5 dəfə yavaş oldu (məsələn, 10000 əməliyyatda 3,3025 ms-ə qarşı 1,3483 ms). Səbəb odur ki, Push zamanı hər dəfə yeni Node obyekti yaradılır və yaddaş ayrılır, Pop isə yalnız top göstəricisini bir addım aşağı çəkir. Bu fərq sabit əmsal xarakterlidir və mürəkkəbliyi dəyişmir, hər iki əməliyyat O(1)-dir.

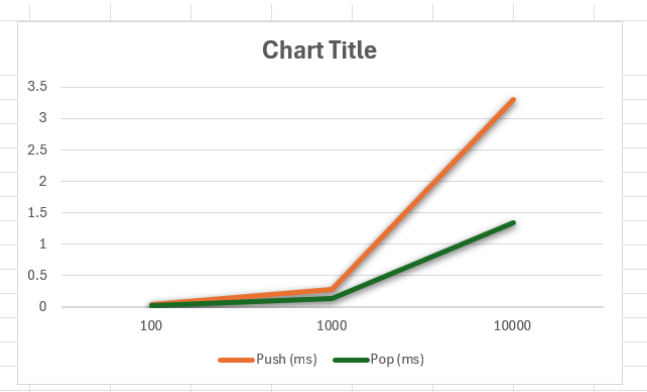
2. Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?
Bəli. Əməliyyat sayı 10 dəfə artdıqda ümumi vaxt da təxminən 10 dəfə artdı, yəni bir əməliyyatın orta vaxtı demək olar ki, dəyişmədi. Push 100 → 1000 → 10000: 0,0339 → 0,2776 → 3,3025 ms, Pop isə 0,0136 → 0,1299 → 1,3483 ms. Bu, hər Push və Pop-un sabit vaxtda, yəni O(1)-də işlədiyini göstərir. 10000 əməliyyatda Push-un artımının 10 dəfədən bir qədər çox olması çoxlu node yaradıldığı üçün garbage collector-un və yaddaş ayrılmasının təsiri ilə izah oluna bilər. Digər kiçik sapmalar ölçmə xətasından və paylaşılan serverin yükündən irəli gəlir

3. Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?
Array əsaslı stack-də hər push üçün yeni node obyekti yaradılmadığı üçün adətən daha sürətli olardı və ardıcıl yaddaş keşə də daha uyğundur. Amma ölçü dolanda array genişləndirilməlidir və həmin anda bir push O(n) vaxt apara bilər. Linked List versiyada isə hər əməliyyat sabit vaxtlıdır, lakin ümumi sürət bir qədər aşağı ola bilər.

4. Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?
İstifadəçinin “geri” davranışı LIFO məntiqinə tam uyğundur: ən son baxılan səhifə ilk geri qaytarılır. Stack-in əsas əməliyyatları O(1)-dir, bu da tarixçənin idarəsini sürətli və sadə edir.

5. Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?
Funksiya çağırışları (call stack) və rekursiya, mətn redaktorlarında Undo/Redo, mötərizələrin balansının yoxlanması, infiks ifadələrin postfiksə çevrilməsi və hesablanması, DFS (dərinlik üzrə axtarış) alqoritmi.

Qrafik



Tapşırıq 3 – Airport Check-in Queue System (Queue)

A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir?

FIFO (First In, First Out) “ilk daxil olan, ilk xidmət alır” deməkdir. Queue-da elementlər rear tərəfdən əlavə olunur (enqueue), front tərəfdən çıxarılır (dequeue). Adi həyatdakı növbəyə tam uyğundur.

2. Queue və Stack arasında əsas fərqlər hansılardır?

Stack LIFO, Queue isə FIFO prinsipi ilə işləyir. Stack-də bütün əməliyyatlar bir ucda (top) gedir. Queue-da isə iki uc var: element rear-dən daxil olur, front-dan çıxır. Stack Undo, rekursiya və tarixçə üçün, Queue isə növbə, planlaşdırma və ardıcıl emal üçün istifadə olunur.

3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?

Sərnişinlər gəliş sırası ilə xidmət almalıdır. Əvvəl gələn sərnişin əvvəl qeydiyyatdan keçməlidir ki, ədalət gözlənilsin. Queue bu qaydanı avtomatik təmin edir. Sərnişin əlavə etmək və ən əvvəldəkini xidmətə çağırmaq O(1) vaxtda həyata keçir.

4. Queue istifadə olunmasa hansı problemlər yarana bilər?

Sıralama pozulardı. Məsələn, Stack istifadə olunsaydı, ən son gələn sərnişin ilk xidmət alardı, əvvəl gələnlər isə uzun müddət gözləməli olardı. Bu, ədalətsizliyə və narazılığa səbəb olardı. Düzgün struktur olmadıqda sıranı izləmək üçün əlavə mürəkkəb məntiq tələb olunardı.

5. Circular Queue nə kimi üstünlüklər təqdim edir?

Adi array əsaslı queue-da elementlər çıxarıldıqca front irəliləyir və başdakı boş yerlər istifadə olunmur. Circular Queue-da array-in sonu başlanğıcla əlaqələndirilir, ona görə boşalan yerlər təkrar istifadə olunur. Nəticədə yaddaşdan daha səmərəli istifadə olunur və elementləri sürüşdürməyə ehtiyac qalmır.

B. Proqramlaşdırma

```
# Hava limani check-in


class Passenger:
    def __init__(self, passenger_id, name, flight_number):  # sernisi yaradilanda 3 deyer gelir
        self.passenger_id = passenger_id  # Passenger ID
        self.name = name  # Name
        self.flight_number = flight_number  # Flight Number



class Node:  # novbenin bir qutusu
    def __init__(self, data):  # qutunun icine Passenger qoyulur
        self.data = data  # bu qutuda saxlanan sernisi
        self.next = None  # sonraki qutuya ox; evvelce yoxdur



class Queue:  # FIFO novbesi
    def __init__(self):  # bos novbe
        self.front = None  # basda (birinci xidmet alacaq) sernisi yoxdur
        self.rear = None  # axirda (en son duran) sernisi yoxdur
        self.count = 0  # novbede nece nefer var

    def is_empty(self):  # novbe bosdurmu?
        return self.front is None  # front yoxdursa bosdur

    def enqueue(self, passenger):  # Add Passenger: sernisini axıra qoy, O(1)
```

```
    new_node = Node(passenger)  # sernisi ucun yeni qutu
    if self.is_empty():  # novbe bosh idise
        self.front = new_node  # hem bas
        self.rear = new_node  # hem axir eyni qutudur
    else:  # novbede artiq sernisi varsa
        self.rear.next = new_node  # kohne axirin oxu yeni qutuya baxsin
        self.rear = new_node  # indi axir yenidir
    self.count += 1  # say 1 artsin

def dequeue(self):  # Check-In: basdakını cixar, O(1)
    if self.is_empty():  # xidmet verilecek sernisi yoxdursa
        return None  # underflow
    passenger = self.front.data  # basdaki sernisini yadda saxla
    self.front = self.front.next  # bas sonraki sernisi olsun
    if self.front is None:  # sonuncu da cixdisa
        self.rear = None  # axir da bosalsin (novbe tamamen bos)
    self.count -= 1  # say 1 azalsin
    return passenger  # check-in olan sernisini qaytar

def peek(self):  # Peek Next: basdakina bax, cixarma, O(1)
    if self.is_empty():  # novbe bosdursa
        return None  # hec ne yoxdur
    return self.front.data  # növbədə birinci sernisi

def display(self):  # Display Queue: basdan axira goster, dequeue etme
    if self.is_empty():  # bosdursa
        print("Novbe bosdur.")  # mesaj
        return  # cix
    current = self.front  # gezmeye basdan basla (FIFO)
    sira = 1  # ekranda 1-ci, 2-ci...
    while current is not None:  # axira qeder
        p = current.data  # bu qutudaki sernisi
        print("Sira:", sira)  # novbedeki yeri
        print("Passenger ID:", p.passenger_id)  # ID
```

```
        print("Name:", p.name)  # ad
        print("Flight Number:", p.flight_number)  # ucus
        print("-----")  # ayirici
        current = current.next  # sonraki sernisi
        sira += 1  # sira nomresi artsin

    def get_count(self):  # Count Passengers
        return self.count  # novbedeki sernisi sayi



def print_passenger(p):  # bir sernisinin 3 sahesini yaz
    print("Passenger ID:", p.passenger_id)  # ID
    print("Name:", p.name)  # ad
    print("Flight Number:", p.flight_number)  # ucus



def main():  # menyu
    novbe = Queue()  # bos check-in novbesi

    while True:  # Exit deyene qeder
        print("\n1. Add Passenger")  # enqueue
        print("2. Check-In Passenger")  # dequeue
        print("3. Peek Next Passenger")  # peek
        print("4. Display Queue")  # hamisi
        print("5. Count Passengers")  # say
        print("6. Exit")  # cix
        secim = input("Secim: ")  # istifadeci nomre yazar

        if secim == "1":  # Add
            passenger_id = input("Passenger ID: ")  # 1-ci sahe
            name = input("Name: ")  # 2-ci sahe
            flight_number = input("Flight Number: ")  # 3-cu sahe
            p = Passenger(passenger_id, name, flight_number)  # sernisi obyekti
            novbe.enqueue(p)  # axira dur
```

```
            print("Sernisi novbeye elave olundu.")  # tesdiq


        elif secim == "2":  # Check-In
            p = novbe.dequeue()  # basdakini cixar
            if p is None:  # novbe bos idise
                print("Novbede sernisi yoxdur.")  # underflow
            else:  # kimse check-in oldu
                print("Check-in olunan sernisi:")  # kim kecdi
                print_passenger(p)  # 3 sahe


        elif secim == "3":  # Peek
            p = novbe.peek()  # basdakina bax, silme
            if p is None:  # bosdursa
                print("Novbede sernisi yoxdur.")
            else:
                print("Novbeti sernisi (birinci duran):")
                print_passenger(p)  # kim növbədə birinci


        elif secim == "4":  # Display
            novbe.display()  # novbeni pozmadan goster


        elif secim == "5":  # Count
            print("Novbedeki sernisi sayi:", novbe.get_count())  # count


        elif secim == "6":  # Exit
            print("Cixilir.")
            break  # while-dan cix


        else:  # 1-6 deyil
            print("Yanlis secim.")



if __name__ == "__main__":  # fayl birbasa ise dusende
```

```
main()  # menyunu ac
```

C. Təcrübə və Eksperiment

Element sayi	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.033	0.0137	0.0078
1000	0.2792	0.1327	0.078
10000	3.4584	1.3874	0.8243

D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur?

Növbəyə sərnişinlər 1, 2, 3 ardıcıllığı ilə əlavə edildi və Dequeue zamanı eyni ardıcıllıqla çıxdı. Yəni ilk əlavə edilən sərnişin ilk qeydiyyatdan keçdi. Bu, Display Queue çıxışında da görünür.

2. Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?

Bəli. Nəzəriyyəyə görə Enqueue, Dequeue və Peek O(1)-dir. Cədvəldəki vaxtlar n əməliyyatın cəmi olduğu üçün element sayı 10 dəfə artanda ümumi vaxt də təxminən 10 dəfə artmalıdır.

3. Queue əvəzinə Stack istifadə olunsa nə baş verərdi?

Sərnişinlərin xidmət sırası tərsinə dönərdi: ən son gələn ilk xidmət alardı. Bu, hava limanı ssenarisinin məntiqinə ziddir, əvvəl gələnlər uzun müddət gözləməli olardı. Mürəkkəblik baxımından fərq olmazdı (hər ikisi O(1)), amma davranış baxımından Stack yararsız olardı.

4. Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?

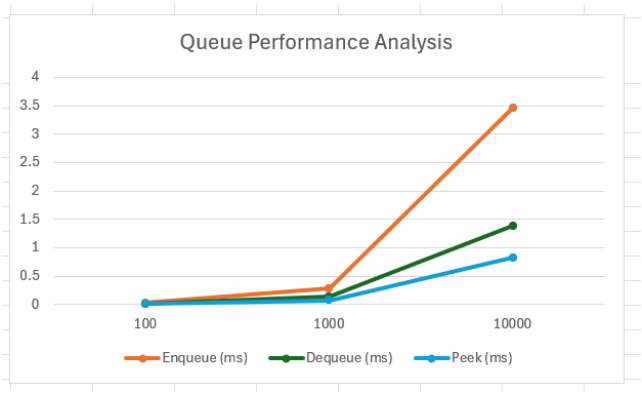
Ümumi vaxt element sayı ilə təxminən mütənasib artdı: Enqueue 0,033 → 0,2792 → 3,4584 ms, Dequeue 0,0137 → 0,1327 → 1,3874 ms, Peek 0,0078 → 0,078 → 0,8243 ms. Hər keçiddə artım təxminən 10 dəfədir, bir əməliyyatın orta vaxtı isə demək olar ki, dəyişmir (məsələn, Enqueue üçün təxminən 0,0003 ms). Enqueue digər iki əməliyyatdan yavaşdır, çünki hər dəfə yeni Node obyekti yaradılır.

5. Circular Queue istifadə etməyin üstünlükləri nələrdir?

Circular Queue-da array-in sonu başlanğıcla əlaqələndirilir, ona görə çıxarılan

elementlərin yerləri təkrar istifadə olunur və elementləri sürüşdürmək lazım olmur. Bu, yaddaşdan daha səmərəli istifadə edir. Eyni zamanda hər dəfə yeni Node obyekti yaratmaq lazım olmadığı üçün Enqueue və Dequeue praktikada daha sürətli ola bilər. Çatışmazlığı ölçünün əvvəlcədən müəyyən olunmasıdır.

Qrafik



8. Analiz

Üç strukturun ümumi müqayisəsi

Struktur	**Əlavə**	**Silmə**	**Axtarış**	**Giriş prinsipi**
Linked List	O(1) (sonda, tail ilə)	O(n) axtarışla birgə	O(n)	İstənilən yerdə
Stack	O(1)	O(1)	O(n)	LIFO
Queue	O(1)	O(1)	O(n)	FIFO

Stack və Queue-da bütün əsas əməliyyatlar O(1)-dir, ona görə ölçmələrdə ümumi vaxt əməliyyat sayı ilə xətti artdı, bir əməliyyatın orta vaxtı isə sabit qaldı. Linked List-də Insert (tail ilə) də eyni qaydada davrandı: onun ümumi vaxtı n sayda əlavənin cəmidir, bir əlavənin vaxtı isə sabitdir. Axtarış və silmə isə siyahını gəzməyi tələb etdiyi üçün tək əməliyyatın vaxtı element sayı ilə xətti artdı.

Sistem parametrlərinin nəticələrə təsiri

Ölçmələr OneCompiler platformasında (Python 3.12.14, Linux, aarch64, 4 nüvə) aparıldığı üçün nəticələrə təsir edən amillər bunlardır:

- Interpretator: Python hər sətri interpretasiya edir və hər node tam bir obyektdir. Buna görə mütləq vaxtlar C++ kimi kompilyasiya olunan dillərdən daha uzun ola bilər. Lakin artım xarakteri (O(1), O(n)) dəyişmir.
- Server və paylaşılan mühit: online server başqa istifadəçilərlə paylaşılır, ona görə eyni kodun vaxtları hər işə salmada fərqlənə bilər. Daha etibarlı nəticə üçün ölçməni bir neçə dəfə təkrarlayıb orta götürmək olar.
- CPU arxitekturası: server ARM64 (aarch64) arxitekturasındadır, bu da x86 prosessorlarla (məsələn, mənim noutbukumdakı Intel Core i5) müqayisədə vaxtlarda fərq yarada bilər.
- Keş: Linked List node-ları yaddaşda səpələndiyi üçün keş buraxılışları (cache miss) çoxdur. Bu, nəzəri O(n) ilə praktik vaxt arasında kiçik fərq yarada bilər.
- Garbage collector: böyük sayda node yaradıldıqda (məsələn, 10000 Enqueue) Python-un garbage collector-u işə düşə bilər və bu, vaxtı bir qədər artırar.
- Taymer dəqiqliyi: kiçik ölçülərdə (100 element) vaxtlar çox qısadır, ona görə taymerin dəqiqliyi nəticəyə nisbətən çox təsir edir.
- Noutbuk parametrləri: HP Pavilion-un CPU, RAM, SSD və Power Mode (Balanced) ayarları ölçmələrə təsir etməyib, çünki kod serverdə icra olunub. Bu parametrlər yalnız kodu yazdığım və nəticələri izlədiyim cihaza aiddir.

9. Nəticə

Bu laboratoriya işində Linked List, Stack və Queue strukturlarını Python dilində sıfırdan implementasiya etdim və üç real ssenari üzərində sınaqdan keçirdim. Ölçmələr nəzəriyyəni böyük ölçüdə təsdiqlədi: Stack və Queue-nun əsas əməliyyatları O(1) vaxtda işlədi (Queue üçün bir əməliyyatın orta vaxtı 100, 1000 və 10000 elementdə demək olar ki, dəyişmədi), Linked List-də axtarış və silmə isə element sayı ilə xətti artdı (O(n)).

Hər strukturun öz məqsədi var. Linked List dinamik ölçü və çevik əlavə/silmə üçün, Stack geri qayıtma və Undo məntiqi üçün, Queue isə ədalətli növbə üçün uyğundur. İş nəticəsində göstəricilər (reference) ilə işləməyi, dinamik yaddaş idarəsini və eksperimental nəticələri Big-O ilə müqayisə etməyi öyrəndim. Həmçinin ölçmə mühitinin, yəni interpretatorun, serverin yükünün və taymer dəqiqliyinin nəticələrə necə təsir etdiyini gördüm.

10. İstinadlar

1. **Weiss, M. A. *Data Structures and Algorithm Analysis in C++*, 4th Edition, Pearson.**
2. **OneCompiler: https://onecompiler.com/python**

AI Usage Declaration

Tool: Claude / Cursor
Purpose: Hesabatın strukturu, araşdırma (A) və analiz (D) hissələrinin mətninin düzəlişləri, kod şərhlərinin genişləndirilməsi və hesabatın yoxlanmasında kömək.
Used Sections: Tapşırıq 1-3 (A, B və D hissələri), Sistem məlumatları, Analiz, Nəticə.